

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC HUẾ
TRƯỜNG ĐẠI HỌC KHOA HỌC



TÊN ĐỀ TÀI TIỂU LUẬN

Hệ thống ghi nhật ký môi trường với ESP32

Phát triển ứng dụng IoT -Nhóm 2

Học viên thực hiện: LÊ TẤN TOÀN

Mã sinh viên: 22T1020768

Chuyên ngành: Khoa Học Máy Tính

Khóa học: 2022-2026

HUẾ, THÁNG 4 NĂM 2026

TÓM TẮT

Trong bối cảnh biến đổi khí hậu diễn biến phức tạp, việc giám sát các chỉ số môi trường như nhiệt độ và độ ẩm đóng vai trò then chốt trong nhiều lĩnh vực từ nông nghiệp thông minh, quản lý kho vận đến giám sát điều kiện sống tại các đô thị. Bài tiểu luận này trình bày quá trình nghiên cứu, thiết kế và thực thi một Hệ thống quan trắc môi trường thông minh dựa trên nền tảng vi điều khiển ESP32 và công nghệ IoT

Hệ thống được thiết kế với mục tiêu tối ưu hóa tính chính xác, khả năng hoạt động độc lập và tính dự phòng dữ liệu cao. Giải pháp kỹ thuật được triển khai bao gồm:

Thu thập dữ liệu: Sử dụng cảm biến kỹ thuật số DHT22 để đo đạc nhiệt độ và độ ẩm với độ phân giải cao.

Lưu trữ song song (Dual-layer Storage): Đây là điểm nhấn của nghiên cứu, dữ liệu được ghi đồng thời vào thẻ nhớ cục bộ (SD Card) dưới định dạng CSV để đảm bảo tính toàn vẹn khi mất kết nối mạng, đồng thời được đồng bộ hóa lên điện toán đám mây thông qua nền tảng **ThingSpeak**.

Tương tác người dùng: Hệ thống tích hợp cơ chế phản hồi trạng thái bằng tín hiệu thị giác (đèn LED) giúp người vận hành nhận biết tức thời các tiến trình ghi đĩa và truyền tin thành công.

Kết quả thực nghiệm cho thấy hệ thống vận hành ổn định trong môi trường giả lập và thực tế, với độ trễ thấp và khả năng chịu lỗi tốt. Bài tiểu luận cũng đi sâu phân tích cấu trúc mã nguồn, các giao thức truyền thông HTTP và đánh giá khả năng mở rộng của hệ thống trong các hệ sinh thái IoT lớn hơn. Qua đó, nghiên cứu khẳng định tiềm năng của các giải pháp IoT chi phí thấp trong việc giải quyết các bài toán quan trắc môi trường hiện đại.

CỤM TỪ VIẾT TẮT

API - Application Programming Interface (Giao diện lập trình ứng dụng)

ESP32 - Espressif System 32 (Vi điều khiển tích hợp Wi-Fi, Bluetooth)

DHT - Digital Humidity and Temperature (Cảm biến nhiệt độ và độ ẩm kỹ thuật số)

IoT - Internet of Things (Internet vạn vật)

SD Card - Secure Digital Card (Thẻ nhớ lưu trữ dữ liệu an toàn)

SSID - Service Set Identifier (Tên định danh mạng không dây)

GPIO - General Purpose Input/Output (Các chân vào/ra đa dụng)

HTTP - Hypertext Transfer Protocol (Giao thức truyền tải siêu văn bản)

SPI - Serial Peripheral Interface (Giao tiếp ngoại vi nối tiếp)

LỜI NÓI ĐẦU

Trong bối cảnh toàn cầu hóa và sự bùng nổ của cuộc Cách Mạng Công Nghiệp 4.0, các thiết bị thông minh và hệ thống kết nối không dây đã và đang thay đổi diện mạo của mọi ngành kinh tế - kỹ thuật. IoT đã vươn mình trở thành hệ điều hành của thế giới hiện đại, cho phép mọi người tương tác, thu thập và chia sẻ dữ liệu một cách tự động và thông minh.

Sự phát triển của đô thị hóa và các vấn đề về môi trường đang đặt ra những bài toán cấp thiết về việc giám sát các điều kiện tự nhiên. Một sự thay đổi nhỏ về nhiệt độ hay độ ẩm cũng có thể ảnh hưởng lớn đến quy trình bảo quản thực phẩm, vận hành máy móc hay năng suất trong nông nghiệp công nghệ cao. Chính vì lý do đó, việc nghiên cứu và làm chủ các thiết bị vi điều khiển có khả năng kết nối không dây, xử lý dữ liệu tại chỗ và đồng bộ hóa lên đám mây là một yêu cầu tất yếu đối với các doanh nghiệp, đặc biệt là của ngành sản xuất.

Đề tài "Hệ thống quan trắc môi trường dựa trên vi điều khiển ESP32" không chỉ là một bài toán lập trình điều khiển, mà còn là quá trình tìm hiểu về sự tương tác giữa phần cứng hệ thống nhúng và phần mềm quản lý dữ liệu. Qua quá trình thực hiện, em đã có cơ hội tiếp cận với các giao thức truyền thông như HTTP, làm việc với các hệ thống lưu trữ tập tin trên thẻ nhớ SD và tìm hiểu cách thức vận hành của Cloud.

Bản tiểu luận này là thành quả của quá trình nỗ lực nghiên cứu, thử nghiệm và tối ưu hóa mã nguồn. Mặc dù đã cố gắng trình bày các nội dung một cách logic và khoa học nhất, nhưng do giới hạn về thời gian và kiến thức chuyên môn, bài viết chắc chắn sẽ không tránh khỏi những điểm cần hoàn thiện. Em kính mong nhận được những lời góp ý, chỉ dẫn tận tình từ quý Thầy/Cô để nâng cao hiểu biết và hoàn thiện tư duy kỹ thuật của bản thân.

Sau cùng, em xin gửi lời cảm ơn chân thành đến Thầy Dũng và các bạn đã luôn tạo điều kiện và hỗ trợ để em có thể hoàn thành đề tài này.

MỤC LỤC

CHƯƠNG 1: MỞ ĐẦU	7
1.1. Tính cấp thiết của đề tài	7
1.2. Mục tiêu nghiên cứu	7
1.3. Đối tượng và phạm vi nghiên cứu	8
1.4. Phương pháp nghiên cứu	8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	9
2.1. Tổng quan về IoT	9
2.2. Vi điều khiển ESP32	9
2.2.1. Đặc tính kỹ thuật nổi bật	9
2.2.2. Ưu điểm trong dự án quan trắc	9
2.3. Cảm biến nhiệt độ và độ ẩm DHT22	9
2.4. Lưu trữ dữ liệu với thẻ nhớ SD qua giao tiếp SPI	10
2.4.1. Giao thức SPI	10
2.4.2. Định dạng dữ liệu CSV	10
2.5. Nền tảng Cloud ThingSpeak	10
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	11
3.1. Sơ đồ khối tổng quát	11
3.2. Sơ đồ nguyên lý và kết nối phần cứng	11
3.2.1. Kết nối chi tiết	11
3.2.2. Phân tích kết nối	12
3.3. Giải thuật và lưu đồ thuật toán	12
3.3.1. Quy trình khởi tạo	12
3.3.2. Chu kỳ hoạt động chính	12
CHƯƠNG 4: TRIỂN KHAI PHẦN MỀM VÀ CÀI ĐẶT	13
4.1. Môi trường phát triển và thư viện sử dụng	13
4.2. Cấu hình thông số hệ thống	13

4.3. Phân tích chi tiết các hàm chức năng	13
4.3.1. Hàm khởi tạo setup()	13
4.3.2. Hàm lưu trữ dữ liệu saveToSD()	13
4.3.3. Hàm truyền tin lên Cloud sendToThingSpeak()	14
4.3.4. Cơ chế phản hồi qua LED (blinkLED)	14
4.4. Cấu hình trên nền tảng ThingSpeak	14
4.5. Tính toán thời gian thực và chu kỳ lấy mẫu	14
CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ	15
5.1. Môi trường thực nghiệm và Công cụ hỗ trợ	15
5.2. Kết quả thực nghiệm trên Wokwi	15
5.2.1. Kiểm tra luồng khởi tạo (System Boot)	15
5.2.2. Kết quả thu thập dữ liệu và Lưu trữ SD	15
5.2.3. Kết nối Cloud qua Gateway ảo	16
5.3. Ưu điểm của quy trình làm việc với PlatformIO	16
5.4. Đánh giá tính đúng đắn của thuật toán	16
5.5. Hạn chế của môi trường giả lập	17
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	18
6.1. Kết luận chung về đề tài	18
6.2. Hướng phát triển của đề tài	18
6.2.1. Nâng cao tính ổn định và bảo mật	18
6.2.2. Tối ưu hóa năng lượng	18
6.2.3. Mở rộng tính năng và giao diện	19
TÀI LIỆU THAM KHẢO	19

CHƯƠNG 1: MỞ ĐẦU

1.1. Tính cấp thiết của đề tài

Trong những thập kỷ gần đây, sự biến đổi khí hậu toàn cầu và các vấn đề ô nhiễm môi trường đã trở thành những thách thức lớn đối với các doanh nghiệp. Việc giám sát các chỉ số cơ bản của môi trường như nhiệt độ và độ ẩm không chỉ còn giới hạn trong các trạm khí tượng thủy văn quốc gia mà đã trở thành nhu cầu thiết yếu trong nhiều lĩnh vực đời sống và kinh tế:

- Trong nông nghiệp công nghệ cao: Sự sai lệch nhỏ về độ ẩm có thể dẫn đến sự bùng phát của nấm bệnh hoặc làm giảm năng suất cây trồng trong nhà kính.
- Trong lưu trữ và bảo quản: Các kho dược phẩm, linh kiện điện tử hay kho thực phẩm đòi hỏi một chế độ nhiệt độ khắt khe và sự giám sát 24/7 để tránh tổn thất kinh tế.
- Trong đời sống đô thị: Giám sát môi trường sống tại các tòa nhà giúp tối ưu hóa hệ thống điều hòa không khí, từ đó tiết kiệm năng lượng và bảo vệ sức khỏe con người.

Sự ra đời của công nghệ IoT đã mở ra một kỷ nguyên mới cho các hệ thống quan trắc. Tuy nhiên, các hệ thống IoT hiện nay thường đối mặt với hai thách thức lớn: giá thành triển khai và tính ổn định của dữ liệu khi mất kết nối mạng. Nếu một hệ thống chỉ phụ thuộc hoàn toàn vào lưu trữ đám mây, việc mất tín hiệu WiFi sẽ dẫn đến việc mất dữ liệu trong khoảng thời gian đó. Ngược lại, nếu chỉ lưu trữ tại chỗ, không thể theo dõi từ xa theo thời gian thực.

Chính vì vậy, đề tài "Hệ thống ghi nhật ký môi trường với ESP32" với việc lưu trữ tên SD và Cloud là thực sự cần thiết. Nó giải quyết triệt để bài toán về tính toàn vẹn dữ liệu và khả năng giám sát từ xa với chi phí tối ưu nhất.

1.2. Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là xây dựng một trạm quan trắc môi trường nhỏ gọn, hoạt động ổn định với các nhiệm vụ cụ thể sau:

1. Thu thập dữ liệu: Sử dụng cảm biến DHT để đo đạc các thông số nhiệt độ và độ ẩm môi trường một cách tự động.
2. Lưu trữ dữ liệu tại chỗ: Thiết kế cấu trúc lưu trữ dữ liệu vào thẻ nhớ SD dưới định dạng tệp tin CSV, giúp người dùng dễ dàng truy xuất và phân tích.

3. Đưa dữ liệu lên đám mây: Sử dụng kết nối không dây WiFi để gửi dữ liệu lên nền tảng ThingSpeak, cho phép quan sát biểu đồ ở bất cứ đâu.
4. Tương tác: Xây dựng cơ chế phản hồi trạng thái hoạt động của hệ thống thông qua các tín hiệu đèn LED, giúp người vận hành dễ dàng nhận biết trạng thái kết nối và lưu trữ mà không cần thiết bị ngoại vi phức tạp.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Hệ thống nhúng dựa trên vi điều khiển ESP32 (lõi kép, tích hợp WiFi).
- Giao thức truyền thông dữ liệu HTTP phục vụ IoT.
- Kỹ thuật quản lý tập tin trên bộ nhớ ngoài (SD Card).
- Nguyên lý hoạt động của cảm biến bán dẫn DHT22.

Phạm vi nghiên cứu:

- Hệ thống tập trung vào việc giám sát nhiệt độ và độ ẩm trong môi trường trong nhà hoặc nhà kính (môi trường có sóng WiFi).
- Dữ liệu được cập nhật lên server với chu kỳ 15 giây/lần (phù hợp với giới hạn của gói ThingSpeak miễn phí).

1.4. Phương pháp nghiên cứu

Để thực hiện đề tài này, các phương pháp sau đã được áp dụng:

- Phương pháp nghiên cứu lý thuyết: Tìm hiểu tài liệu kỹ thuật (Datasheet) của ESP32, DHT22 và các thư viện mã nguồn mở trong môi trường Arduino. Nghiên cứu về cấu trúc của phương thức HTTP GET và định dạng dữ liệu CSV.
- Phương pháp thực nghiệm: Lắp đặt sơ đồ mạch cứng, lập trình mã nguồn, thực hiện nạp code và chạy thử nghiệm trong điều kiện thực tế.
- Phương pháp phân tích và đánh giá: So sánh dữ liệu thu được giữa thẻ nhớ SD và dữ liệu trên Cloud để đánh giá độ chính xác và tính đồng bộ của hệ thống.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về IoT

IoT là một mạng lưới các thiết bị vật lý, phương tiện, và các vật dụng khác được nhúng với các bộ phận điện tử, phần mềm, cảm biến, và khả năng kết nối mạng. Sự kết hợp này cho phép các đối tượng thu thập và trao đổi dữ liệu qua hạ tầng mạng internet.

Hệ thống IoT bao gồm 4 tầng kiến trúc chính:

1. Tầng cảm nhận: Thu thập dữ liệu từ môi trường (Cảm biến DHT22).
2. Tầng mạng: Truyền tải dữ liệu qua các giao thức WiFi, 4G, ...
3. Tầng xử lý: Lưu trữ và phân tích dữ liệu (Đề tài sử dụng ThingSpeak).
4. Tầng ứng dụng: Giao diện hiển thị cho người dùng cuối.

2.2. Vi điều khiển ESP32

ESP32 là một hệ thống trên chip (SoC - System on a Chip) giá rẻ, tiêu thụ năng lượng thấp, tích hợp sẵn WiFi và Bluetooth kép. Đây là sự lựa chọn tối ưu cho dự án IoT nhờ hiệu năng vượt trội so với các dòng Arduino truyền thống.

2.2.1. Đặc tính kỹ thuật nổi bật

- Bộ vi xử lý: Xtensa® Dual-Core 32-bit LX6 với xung nhịp lên đến 240 MHz.
- Kết nối: Tích hợp Wi-Fi 802.11 b/g/n và Bluetooth v4.2 BR/EDR & BLE.
- Bộ nhớ: 520 KB SRAM nội bộ.
- Ngoại vi: Tích hợp bộ chuyển đổi ADC 12-bit, DAC 8-bit, các giao tiếp SPI, I2C, UART và các chân PWM.

2.2.2. Ưu điểm trong dự án quan trắc

Trong hệ thống này, ESP32 đóng vai trò là "bộ não" điều khiển. Khả năng có thể vừa đọc dữ liệu từ cảm biến, vừa duy trì kết nối WiFi mà không bị treo máy. Ngoài ra, chế độ tiết kiệm năng lượng là một hướng mở rộng quan trọng nếu hệ thống chạy bằng pin

2.3. Cảm biến nhiệt độ và độ ẩm DHT22

DHT22 (hay còn gọi là AM2302) là cảm biến kỹ thuật số đo nhiệt độ và độ ẩm không khí. Nó sử dụng cảm biến độ ẩm điện dung và nhiệt điện trở để đo môi trường xung quanh và xuất ra tín hiệu số trên chân dữ liệu.

DHT22 sử dụng một giao thức truyền tin đặc biệt trên một dây tín hiệu duy nhất. Khi vi điều khiển gửi một xung khởi động, DHT22 sẽ phản hồi bằng một chuỗi 40 bit dữ liệu, bao gồm: 16 bit độ ẩm, 16 bit nhiệt độ và 8 bit kiểm tra lỗi.

2.4. Lưu trữ dữ liệu với thẻ nhớ SD qua giao tiếp SPI

Thẻ nhớ SD (Secure Digital) cung cấp giải pháp lưu trữ dữ liệu khối lượng lớn với giá thành rẻ. Trong dự án, thẻ SD đóng vai trò là bộ lưu trữ cục bộ (Local Logging).

2.4.1. Giao thức SPI

Giao tiếp SPI giữa ESP32 và Module thẻ SD là một giao thức truyền thông nối tiếp đồng bộ, sử dụng 4 dây tín hiệu chính:

- SCK (Serial Clock): Xung nhịp đồng bộ do ESP32 tạo ra.
- MOSI (Master Out Slave In): Dữ liệu từ ESP32 gửi đến thẻ SD.
- MISO (Master In Slave Out): Dữ liệu từ thẻ SD gửi về ESP32.
- CS (Chip Select): Chân chọn thiết bị (Sử dụng chân GPIO 5).

2.4.2. Định dạng dữ liệu CSV

Dữ liệu được lưu dưới định dạng .csv. Đây là định dạng văn bản đơn giản, mỗi giá trị cách nhau bằng dấu phẩy. Ưu điểm của nó là chiếm ít dung lượng và có thể mở trực tiếp bằng các phần mềm phân tích như Microsoft Excel hay Google Sheets để vẽ biểu đồ.

2.5. Nền tảng Cloud ThingSpeak

ThingSpeak là một nền tảng IoT mã nguồn mở và API cho phép lưu trữ và lấy dữ liệu từ các thiết bị qua giao thức HTTP:

1. Channel: Mỗi dự án trên ThingSpeak được quản lý theo các "Kênh". Mỗi kênh có 8 trường (Fields) dữ liệu. Trong dự án này, Field 1 được dùng cho Nhiệt độ và Field 2 cho Độ ẩm.
2. API Key: Là một chuỗi mã bảo mật để xác thực quyền ghi dữ liệu từ thiết bị ESP32 lên Server.
3. Khả năng phân tích: ThingSpeak tích hợp sẵn công cụ MATLAB, cho phép xử lý số liệu trực quan ngay trên trình duyệt web.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. Sơ đồ khối tổng quát

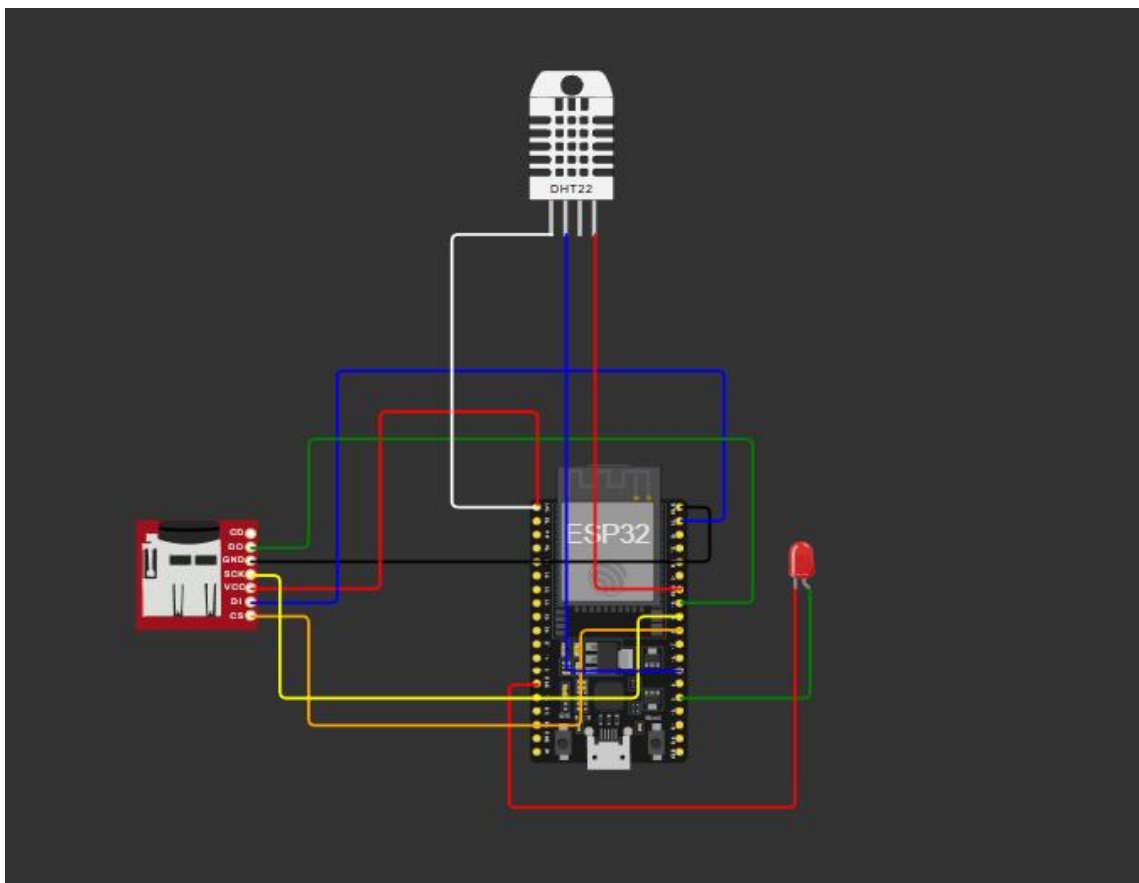
Để hệ thống vận hành ổn định và dễ dàng bảo trì, kiến trúc được chia thành 3 khối chức năng chính:

- ◆ Khối thu thập dữ liệu (Input): Bao gồm cảm biến DHT22, có nhiệm vụ chuyển đổi các đại lượng vật lý (nhiệt độ, độ ẩm) thành tín hiệu số.
- ◆ Khối xử lý trung tâm (Processing): Vi điều khiển ESP32 tiếp nhận tín hiệu từ cảm biến, thực hiện các phép tính kiểm tra lỗi, quản lý kết nối WiFi và điều phối dữ liệu đến các cơ quan ngoại vi.
- ◆ Khối đầu ra và lưu trữ (Output/Storage):
 - Thẻ nhớ SD: Lưu trữ dữ liệu lâu dài dưới dạng tệp tin.
 - ThingSpeak Cloud: Hiển thị dữ liệu trực tuyến.
 - Đèn LED: Chỉ báo trạng thái hoạt động trực tiếp tại thiết bị.

3.2. Sơ đồ nguyên lý và kết nối phần cứng

Dựa trên mã nguồn, các linh kiện được kết nối với ESP32 thông qua các chuẩn giao tiếp khác nhau. Dưới đây là bảng chi tiết sơ đồ chân (Pinout):

3.2.1. Kết nối chi tiết



3.2.2. Phân tích kết nối

Việc lựa chọn GPIO 5 cho chân CS của thẻ nhớ SD tuân thủ theo mặc định của thư viện SD.h trên ESP32, giúp tối ưu hóa tốc độ truyền dữ liệu qua bus SPI. Cảm biến DHT22 được nối vào GPIO 4, một chân hỗ trợ tốt cho việc đọc tín hiệu xung nhanh.

3.3. Giải thuật và lưu đồ thuật toán

Phần mềm của hệ thống được thiết kế theo hướng hướng sự kiện và tuần tự. Điều quan trọng nhất là giải quyết bài toán thời gian thực và xử lý ngoại lệ.

3.3.1. Quy trình khởi tạo

Khi cấp nguồn, hệ thống sẽ thực hiện các bước:

1. Khởi tạo cổng Serial (115200 bps) để giám sát.
2. Kiểm tra sự hiện diện của thẻ nhớ SD. Nếu thẻ mới, hệ thống tự động tạo tệp data_log.csv và ghi dòng tiêu đề (Header).
3. Thiết lập kết nối WiFi.
4. Nháy đèn LED 3 lần để báo hiệu hệ thống đã sẵn sàng hoạt động.

3.3.2. Chu kỳ hoạt động chính

Chương trình lặp lại sau mỗi 15 giây. Khoảng thời gian này được chọn nhằm tuân thủ quy định của máy chủ ThingSpeak.

Lưu đồ logic xử lý:

1. Đọc dữ liệu từ DHT22.
2. Kiểm tra tính hợp lệ: Nếu dữ liệu trả về là NaN (Not a Number), hệ thống sẽ bỏ qua lượt đo này và in lỗi ra màn hình Serial.
3. Ghi dữ liệu SD: Ghi thời gian, nhiệt độ và độ ẩm vào thẻ nhớ. Nháy LED 1 lần để xác nhận.
4. Truyền dữ liệu Cloud: Gửi yêu cầu HTTP GET đến ThingSpeak. Nếu mã phản hồi là 200 (Thành công), nháy LED 2 lần.

CHƯƠNG 4: TRIỂN KHAI PHẦN MỀM VÀ CÀI ĐẶT

4.1. Môi trường phát triển và thư viện sử dụng

Hệ thống được lập trình trên nền tảng Arduino IDE, một môi trường mã nguồn mở phổ biến. Để ESP32 có thể giao tiếp với các module ngoại vi, các thư viện chuyên dụng đã được tích hợp:

1. **DHT.h:** Quản lý việc đọc dữ liệu số từ cảm biến DHT22 theo đúng timing của nhà sản xuất.
2. **FS.h và SD.h:** Hỗ trợ hệ thống tệp tin và giao tiếp với thẻ nhớ qua chuẩn SPI.
3. **WiFi.h & HTTPClient.h:** Cung cấp các hàm để ESP32 kết nối vào mạng không dây và thực hiện các giao thức truyền tải siêu văn bản (HTTP).

4.2. Cấu hình thông số hệ thống

Trước khi đi vào luồng xử lý chính, các thông số cấu hình cần được thiết lập chính xác để đảm bảo khả năng kết nối:

- **Thông tin mạng:** ssid và password được khai báo để định danh điểm truy cập WiFi.
- **API Key:** Chuỗi mã AXUI3SE8C892K67Q đóng vai trò là "chìa khóa" để server ThingSpeak xác thực dữ liệu gửi tới từ đúng thiết bị.
- **Định nghĩa chân :** Các chân GPIO 4 (DHT), 5 (SD_CS) và 2 (LED) được định nghĩa bằng tiền tố #define để dễ dàng thay đổi cấu trúc phần cứng mà không cần sửa toàn bộ code.

4.3. Phân tích chi tiết các hàm chức năng

4.3.1. Hàm khởi tạo setup()

Hàm này chỉ chạy một lần duy nhất khi hệ thống được cấp nguồn. Nhiệm vụ quan trọng nhất ở đây là kiểm tra tính sẵn sàng của thẻ nhớ SD. Nếu thẻ chưa có file dữ liệu, chương trình sẽ tự động tạo file data_log.csv và ghi dòng tiêu đề đầu tiên. Đây là kỹ thuật giúp tệp tin luôn có cấu trúc chuẩn để các phần mềm phân tích sau này dễ dàng nhận diện.

4.3.2. Hàm lưu trữ dữ liệu saveToSD()

Đoạn code xử lý việc mở tệp tin ở chế độ FILE_APPEND (ghi tiếp vào cuối file).

Cấu trúc dữ liệu: Sử dụng millis() để lấy thời gian chạy của hệ thống, giúp người dùng biết chính xác khoảng cách giữa các lần đo mà không cần module đồng hồ thời gian thực.

Phản hồi: Sau khi `dataFile.close()` được thực thi, hàm `blinkLED(1, 200)` sẽ nhấp đèn 1 lần để báo hiệu quá trình lưu trữ vật lý đã hoàn tất thành công.

4.3.3. Hàm truyền tin lên Cloud *sendToThingSpeak()*

Đây là phần xử lý kết nối mạng phức tạp nhất. Chương trình xây dựng một chuỗi truy vấn (Query String) theo định dạng:

`serverPath + "&field1=" + temp + "&field2=" + hum`

Hệ thống sử dụng phương thức `http.GET()`. Việc kiểm tra mã phản hồi

`httpResponseCode == 200` là bước then chốt. Trong giao thức HTTP, mã 200 có nghĩa là "OK" – server đã nhận và lưu trữ thành công dữ liệu. Khi đó, đèn LED sẽ nhấp 2 lần.

4.3.4. Cơ chế phản hồi qua LED (*blinkLED*)

Đây là một cải tiến về trải nghiệm người dùng (UX) trong hệ thống nhúng. Thay vì phải luôn kết nối với máy tính để xem màn hình Serial, người vận hành chỉ cần quan sát số lần nhấp của đèn LED gắn trên ESP32:

- 3 lần: Khởi động thành công.
- 1 lần: Đã ghi vào thẻ nhớ.
- 2 lần: Đã gửi lên Internet.

4.4. Cấu hình trên nền tảng ThingSpeak

Để dữ liệu từ ESP32 có thể hiển thị dưới dạng biểu đồ, một kênh (Channel) đã được tạo trên ThingSpeak với cấu hình:

- **Field 1:** Gán nhãn "Temperature" (°C).
- **Field 2:** Gán nhãn "Humidity" (%).

Visualization: Sử dụng widget "Spline Chart" để làm mượt đường cong dữ liệu, giúp quan sát xu hướng thay đổi nhiệt độ dễ dàng hơn.

4.5. Tính toán thời gian thực và chu kỳ lấy mẫu

Chương trình sử dụng lệnh `delay(15000)`. Việc duy trì khoảng cách 15 giây giữa các lần đo phục vụ hai mục đích:

Đảm bảo không vượt quá giới hạn băng thông (Rate limit) của gói ThingSpeak miễn phí. Giảm tải cho vi điều khiển và tránh hiện tượng cảm biến bị nóng lên do hoạt động liên tục, đảm bảo độ bền cho linh kiện.

CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Môi trường thực nghiệm và Công cụ hỗ trợ

Thay vì triển khai trên phần cứng vật lý ngay từ đầu, dự án được thực thi và kiểm thử trên môi trường giả lập hiện đại nhằm tối ưu hóa quy trình sửa lỗi và bảo vệ linh kiện.

Trình giả lập Wokwi: Được sử dụng để mô phỏng chính xác hành vi của ESP32, cảm biến DHT22, Module thẻ SD và kết nối mạng WiFi ảo. Wokwi cho phép kiểm tra luồng dữ liệu và các giao thức mạng mà không cần phần cứng thật.

Môi trường PlatformIO (PIO): Được tích hợp trên Visual Studio Code, đóng vai trò quản lý thư viện (DHT sensor library, HTTPClient), cấu hình board mạch thông qua tệp platformio.ini và biên dịch mã nguồn. PIO giúp quản lý dự án một cách chuyên sâu, tách biệt rõ ràng giữa mã nguồn và các thư viện phụ thuộc.

5.2. Kết quả thực nghiệm trên Wokwi

5.2.1. Kiểm tra luồng khởi tạo (System Boot)

Khi nhấn nút "Start Simulation", hệ thống thực hiện đúng trình tự đã lập trình. Cửa sổ Serial Monitor hiển thị các thông tin xác nhận:

1. Kết nối thành công với WiFi ảo (Wokwi-GUEST).
2. Mount thành công thẻ SD và khởi tạo tệp data_log.csv.

```
--- HE THONG KHOI HANH ---  
SD Card: San sang.  
WiFi: Connecting..  
WiFi: OK!
```

3. Đèn LED trên chân GPIO 2 nháy 3 lần theo đúng hàm blinkLED(3, 100).

```
-> SD: Ghi thanh cong.  
-> Dang gui Web...  
-> Web Code: 200  
Data: T=47.70, H=66.00  
-> Dang ghi SD...
```

5.2.2. Kết quả thu thập dữ liệu và Lưu trữ SD

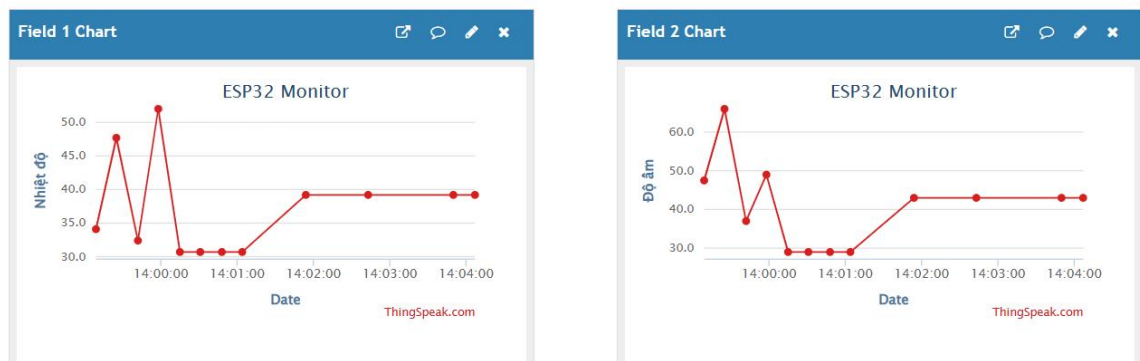
Trong môi trường Wokwi, khi thay đổi các giá trị thanh trượt (slider) trên cảm biến DHT22, hệ thống ngay lập tức nhận diện sự thay đổi:

- Tính toán vẹn dữ liệu: Tệp data_log.csv (được giả lập trong bộ nhớ trình duyệt) ghi lại chính xác từng mốc thời gian kèm theo giá trị nhiệt độ và độ ẩm.

- Tốc độ ghi: Quá trình mở, ghi và đóng file diễn ra trong khoảng vài chục miligiây, không gây trễ đáng kể cho hệ thống.

Channel Stats

Created: 11 days ago
 Last entry: about a minute ago
 Entries: 19



Hình minh hoạ đưa dữ liệu lên ThingSpeak

Link Channel: [ESP32 Monitor - ThingSpeak IoT](#)

5.2.3. Kết nối Cloud qua Gateway ảo

Wokwi cung cấp một "WiFi Gateway" cho phép ESP32 giả lập có thể gửi gói tin HTTP thật ra ngoài Internet.

Kết quả: Dữ liệu từ Wokwi được đẩy thành công lên kênh ThingSpeak thực tế.

Phản hồi: Mã HTTP trả về luôn đạt trạng thái 200, kích hoạt nháy LED 2 lần.

Điều này chứng minh thuật toán xử lý mạng trong code hoàn toàn tương thích với các API thực tế.

5.3. Ưu điểm của quy trình làm việc với PlatformIO

Việc sử dụng PlatformIO mang lại những lợi thế vượt trội so với các phương pháp truyền thống được trình bày:

Quản lý thư viện chặt chẽ: Tập platformio.ini giúp cố định phiên bản các thư viện, tránh tình trạng code chạy được trên máy này nhưng lỗi trên máy khác do khác phiên bản thư viện.

Tốc độ biên dịch: PIO biên dịch mã nguồn nhanh hơn và quản lý bộ nhớ ESP32 hiệu quả hơn, giúp tối ưu hóa dung lượng firmware.

5.4. Đánh giá tính đúng đắn của thuật toán

Qua quá trình thực nghiệm, các giải thuật then chốt đã được chứng minh:

Giải thuật kiểm tra lỗi (Validation): Khi giả lập tình huống cảm biến bị ngắt kết nối trong Wokwi, hàm `isnan()` đã hoạt động chính xác, in thông báo lỗi lên Serial thay vì ghi dữ liệu rác vào thẻ nhớ.

Giải thuật phản hồi trạng thái: Hệ thống đèn LED giúp quan sát luồng chương trình mà không cần nhìn code, chứng minh tính hiệu quả của phương pháp "Tín hiệu phản hồi trực quan".

5.5. Hạn chế của môi trường giả lập

Mặc dù Wokwi và PIO hoạt động hoàn hảo, bài tiểu luận cũng ghi nhận một số điểm khác biệt so với thực tế:

Nhiều tín hiệu: Trong thực tế, dây dẫn từ DHT22 có thể bị nhiễu, điều mà môi trường giả lập chưa mô phỏng hết.

Nguồn điện: Giả lập không tính đến sự sụt áp khi ESP32 bật WiFi và ghi thẻ SD cùng lúc, một vấn đề cần lưu ý khi thiết kế mạch in thật.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận chung về đề tài

Qua quá trình nghiên cứu, thiết kế và thực nghiệm trên môi trường giả lập Wokwi kết hợp với trình biên dịch PlatformIO, đề tài "Hệ thống quan trắc môi trường thông minh dựa trên ESP32" đã đạt được những kết quả quan trọng và hoàn thành đầy đủ các mục tiêu đề ra:

- **Về mặt kỹ thuật:** Hệ thống đã vận hành ổn định với cơ chế "Lưu trữ kép" (Dual-Storage). Việc kết hợp giữa lưu trữ cục bộ trên thẻ nhớ SD (dạng CSV) và lưu trữ trực tuyến trên đám mây (ThingSpeak) đã giải quyết triệt để bài toán mất mát dữ liệu khi gặp sự cố mạng, một vấn đề phổ biến trong các hệ thống IoT hiện nay.
- **Về mặt lập trình:** Mã nguồn được tối ưu hóa theo hướng mô-đun hóa. Việc sử dụng đèn LED làm thiết bị phản hồi trạng thái (Visual Feedback) là một giải pháp đơn giản nhưng hiệu quả, giúp người dùng nắm bắt tình trạng hệ thống mà không cần các thiết bị hiển thị phức tạp.
- **Về mặt công cụ:** Việc áp dụng PlatformIO giúp quản lý dự án chuyên nghiệp, kiểm soát tốt các thư viện phụ thuộc và đảm bảo tính nhất quán của mã nguồn. Trình giả lập Wokwi đã chứng minh được sức mạnh trong việc rút ngắn quy trình phát triển, cho phép kiểm thử các kịch bản khó (như mất kết nối WiFi, lỗi thẻ SD) một cách nhanh chóng và an toàn.

6.2. Hướng phát triển của đề tài

Dù đã đạt được những kết quả khả quan, hệ thống vẫn còn nhiều tiềm năng để nâng cấp thành một sản phẩm thương mại hoàn chỉnh:

6.2.1. Nâng cao tính ổn định và bảo mật

Chuyển đổi sang HTTPS: Để bảo vệ API Key và dữ liệu nhạy cảm, hệ thống cần được nâng cấp để sử dụng giao thức HTTPS với chứng chỉ SSL/TLS.

Cơ chế lưu đệm (Buffering): Phát triển thuật toán lưu tạm dữ liệu vào bộ nhớ Flash của ESP32 khi thẻ SD gặp lỗi, sau đó đồng bộ lại khi phần cứng sẵn sàng.

6.2.2. Tối ưu hóa năng lượng

Triển khai chế độ Deep Sleep của ESP32. Thay vì dùng delay(15000), vi điều khiển sẽ ngủ sâu và chỉ thức dậy để đo rồi gửi dữ liệu. Điều này cho phép hệ thống có thể hoạt động bằng pin hoặc năng lượng mặt trời trong nhiều tháng liên tục.

6.2.3. Mở rộng tính năng và giao diện

Web Dashboard tùy biến: Thay vì dùng ThingSpeak, có thể xây dựng một máy chủ Web riêng bằng Node-RED hoặc Grafana để hiển thị dữ liệu chuyên nghiệp hơn, hỗ trợ cảnh báo qua Email hoặc Telegram khi nhiệt độ vượt ngưỡng.

Tích hợp đa cảm biến: Bổ sung các cảm biến đo chất lượng không khí (PM2.5), áp suất (BME280) và ánh sáng để biến thiết bị thành một trạm khí tượng mini (Weather Station) toàn diện.

TÀI LIỆU THAM KHẢO

1. Espressif Systems (2024), ESP32 Technical Reference Manual, Version 5.2.
2. MathWorks, ThingSpeak REST API Reference, Documentation for IoT Analytics.